

RACE Tutor AI

Classical ML Reading Comprehension Tutor

Abdullah Sajid – 22i-2346
Huzaifa Khalid – 22i-2332

May 2026

Presentation Roadmap

- 1 Project Overview
- 2 Dataset and EDA
- 3 Statistical Analysis
- 4 Preprocessing
- 5 Model Selection and Training
- 6 Testing and Results
- 7 Model B and Demo
- 8 Conclusion

Project Goal

Objective

Build a fully running reading-comprehension tutor on the RACE dataset.

- Load and preprocess RACE-style multiple-choice questions.
- Train **Model A** to verify and rank answer options.
- Build **Model B** to generate distractors and graduated hints.
- Provide a Streamlit app for quiz practice, hints, predictions, and analytics.

Final app: `frontend/app.py`

Best Model A artifact: `models/model_a/logreg_candidate_rich.pkl`

Backend Implementation Map

File	Purpose
preprocessing.py	Load, flatten, clean, split, and build candidate examples.
feature_engineering.py	Sparse text features, dense lexical features, cosine features, scaling.
model_a.py	Model builders, training helpers, answer prediction.
train_model_a.py	CLI for Model A training/evaluation.
model_b.py	Extractive distractor and hint generation.
inference.py	End-to-end quiz construction for the frontend.
frontend/app.py	Streamlit user interface.

Dataset Explanation

- Dataset: **RACE**, a reading-comprehension dataset based on English exams.
- Each row contains:
 - article/passage
 - question
 - four options: A, B, C, D
 - correct answer label
- Local expected format:

`id, article, question, A, B, C, D, answer`

HuggingFace Flattening

The ehovy/race format stores options as a list. The backend flattens this into explicit columns A, B, C, and D, keeping only examples with exactly four options.

EDA: Dataset Size and Answer Distribution

Split	Rows	A	B	C	D
Train	70,292	15,317	18,181	19,112	17,682
Validation	8,787	1,914	2,273	2,390	2,210
Test	8,787	1,915	2,272	2,389	2,211

- Distribution is not perfectly uniform.
- In all splits, option C is most frequent and option A is least frequent.
- Train percentages: A 21.8%, B 25.9%, C 27.2%, D 25.2%.

EDA: Text Length Statistics

Split	Article Mean/Median	Question Mean/Median	Option Mean/Median
Train	275.2 / 279	10.0 / 10	5.7 / 5
Validation	274.4 / 278	10.0 / 10	5.7 / 6
Test	273.7 / 278	10.0 / 10	5.7 / 5

Interpretation

Articles are much longer than questions and options. This supports using sparse text features and similarity features rather than raw text length alone.

Statistical Analysis Used

- 1 **Descriptive statistics:** row counts, word-length means, medians.
- 2 **Class distribution analysis:** answer label frequencies.
- 3 **Chi-square goodness-of-fit test:** tested whether answer labels are uniformly distributed.
- 4 **Candidate feature comparison:** compared correct vs incorrect candidate overlap statistics.

Split	Chi-square statistic	p-value
Train	446.12	2.26×10^{-96}
Validation	56.12	3.96×10^{-12}
Test	55.63	5.03×10^{-12}

Interpretation of Statistical Analysis

- The chi-square results show answer labels are statistically different from a perfectly uniform distribution.
- Option A appears less frequently than the other labels.
- This explains why the early option-A-only binary model was limited.
- The final model therefore trains on **all four candidates** instead of only option A.

Correct vs Incorrect Candidate Evidence

On the training split:

- Correct option coverage in article: 0.6814
- Incorrect option coverage in article: 0.6344
- Correct options are slightly longer on average: 6.05 vs 5.76 words

Complete Data Preprocessing Steps

- 1 Load CSV splits or HuggingFace RACE.
- 2 Flatten nested options into A/B/C/D columns.
- 3 Keep only examples with exactly four options.
- 4 Lowercase all text.
- 5 Remove punctuation and normalize whitespace.
- 6 Fill missing text with empty strings.
- 7 Drop invalid answer labels.
- 8 Drop rows with empty articles or questions.
- 9 Expand each question into candidate-level examples.
- 10 Fit vectorizer/scaler on train only; reuse on validation/test.

Sparse Features

- CountVectorizer
- Binary bag-of-words
- Unigrams + bigrams
- Max features: 50,000
- Minimum document frequency: 2

Dense Features

- Length features
- Lexical overlap ratios
- TF-IDF cosine similarities
- Negation flags
- Per-question relative option features
- StandardScaler fitted on train only

Model Selection Process

Model Tried	Observation
Option-A logistic model	Simple baseline but ignores B/C/D training signal.
Candidate logistic model	Better aligned with answer selection; trains all four options.
Candidate SVM	Similar but slightly weaker/slower in slice experiments.
Rich candidate logistic model	Best validation and test answer-selection accuracy.

Final Selection

Logistic Regression was selected because it is transparent, stable on sparse text features, fast enough for this dataset, and produced the best measured result.

Model Training Process

- Final training command:

```
.venv/bin/python -m backend.train_model_a \  
  --train data/raw/train.csv \  
  --val data/raw/val.csv \  
  --test data/raw/test.csv \  
  --model logreg_candidate \  
  --class-weight none \  
  --ngram-max 2 \  
  --c 0.3 \  
  --output-name logreg_candidate_rich
```

- Saved model: models/model_a/logreg_candidate_rich.pkl
- Training examples: one candidate row per option.
- Inference: score A/B/C/D and select highest score.

Testing and Evaluation Setup

- Validation and test sets were held out from training.
- Vectorizers and scalers were fitted only on train.
- Two evaluation views were used:
 - **Binary candidate metrics:** each candidate is correct/incorrect.
 - **Four-option answer selection:** highest-scoring option is compared with the true answer.
- The app task uses four-option answer selection, so that is the primary metric.

Model A Result Progression

Model	Validation Accuracy	Test Accuracy
Option-A binary model	0.3283	0.3214
Candidate model	0.3530	0.3570
Rich candidate model	0.3809	0.3781

Interpretation

Moving from option-A-only training to all-candidate training improved ranking. Adding bigrams, TF-IDF similarity, negation, and relative option features improved it further.

Final Performance Metrics

Split	Binary Acc.	Macro F1	Answer Acc.	Answer F1
Validation	0.7446	0.4671	0.3809	0.3798
Test	0.7444	0.4660	0.3781	0.3771

- Binary accuracy is high because most candidate options are incorrect.
- Macro F1 reveals the imbalance challenge.
- Answer-selection accuracy is the most meaningful app metric.
- Test accuracy improved from 32.14% to 37.81%.

Distractor Generation

- Extracts passage phrases.
- Removes stopwords.
- Filters correct-answer duplicates.
- Ranks by frequency and question relevance.
- Returns three distinct distractors.

Hint Generation

- Ranks supporting sentences.
- Hint 1: broad focus.
- Hint 2: masked evidence.
- Hint 3: direct supporting sentence.

Streamlit Demo Flow

- 1 Open the app:

```
streamlit run frontend/app.py
```

- 2 Load a random RACE sample from `data/raw/test.csv`.
- 3 Answer the question in the Quiz View tab.
- 4 Compare your answer with Model A's predicted answer.
- 5 Reveal Model B hints one at a time.
- 6 View generated distractors.
- 7 Check session analytics: attempts, accuracy, latency, and history.

Conclusion

- The project is fully running end to end.
- Model A uses a classical ML candidate-ranking pipeline.
- Model B provides extractive distractors and graduated hints.
- The Streamlit frontend integrates quiz loading, prediction, hints, and analytics.
- Final test answer-selection accuracy: **37.81%**.

Future Improvements

Direct ranking objectives, stronger semantic features, transformer embeddings, and improved answer-type matching for Model B could further improve performance.

Thank You

Questions?